

Module 2

This is a single, concatenated file, suitable for printing or saving as a PDF for offline viewing. Please note that some animations or images may not work.

Module 2 Study Guide and Deliverables

January 23 – January 29

Theme: Attacks and Defense

Topics:

- Malware: virus, worms, trojan horse, rootkit, zombie, bot, botnet, ransomware,
- Bug: buffer overflow, integer overflow, TOCTTOU, covert channel
- Security model: threat model, trust model, trusted computing base
- Security principles and countermeasures

Readings:

- Chapter 3

Assignments:

- Assignment 1 due **Tuesday, January 30 at 6:00 AM ET**
- Lab 2 due **Tuesday, January 30 at 6:00 AM ET**

Access at "Assignments" on the left-hand course menu.

Assessments:

- Quiz 2 due **Tuesday, January 30 at 6:00 AM ET** (Access at "Assessments" on the left-hand course menu.)

Discussions:

- None

Live Classroom: Join the live classroom and the facilitator's live office hour session at "Live Classroom/Offices" > "Live Classroom" > Launch Meeting.

- Lecture: **Tuesday, January 23, 7:30-9:00 PM ET**
- Facilitator office hour: TBD

Learning Outcomes

By the end of this module, you will be able to:

- Describe vulnerabilities in various system components.
- Describe different types of adversaries and possible attacks.
- Describe different types of malware.
- Describe important cybersecurity principles and common pitfalls.
- Describe security controls and modeling.

Introduction

In Module 1, we introduced the following terms in cybersecurity:

- **Vulnerability:** a vulnerability is a weakness in an information system, system security procedure, internal control, or implementation that could be exploited or triggered by a threat source.
- **Threat:** a threat is any circumstance or event with the potential to adversely impact organizational operations (including mission, functions, image, or reputation), organizational assets, individuals, other organizations, or the nation through an information system via unauthorized access, destruction, disclosure, modification of information, and/or denial of service. It may also refer to the potential for a threat source to successfully exploit a particular information system vulnerability. Threat sources include malware such as spyware, adware, worms, viruses, and attackers.
- **Exploit:** an exploit is a tool or method by which an attacker uses a vulnerability to cause damage to the target.
- **Attack:** an attack is any kind of malicious activity that attempts to collect, disrupt, deny, degrade, or destroy information system resources or the information itself. It usually occurs when a threat source attempts to exploit a vulnerability. A compromise occurs when an attack is successful. It is typically associated with taking over/altering resources
- **Security control:** a security control is a countermeasure. It may be an action, a device, a procedure or a technique that can help block attacks.

In this module, we will explain these terms in more detail. We will also introduce common attacks and defense techniques.

■ Topic 1 Attacks

Adversaries

There are different types of adversaries with various motivations; some adversaries may attack a target by chance, while others may be aimed at a specific target. Usually, we imagine attacks as coming from outside of the organization. However, adversaries can be either **insiders** or **outsiders**. Insider threats may come from disgruntled current or previous employees, third-party employees, or even industrial spies, who usually have direct access to sensitive information or privileged accounts within the network of an organization. They can abuse that access to carry out an insider attack.

Insider attacks are particularly dangerous and may be harder to detect. The total average cost of insider-related incidents is \$11.45 million according to the 2020 Cost of Insider Threats Global Report by the Ponemon Institute. Insider attacks can lead to a variety of consequences, from penalties for non-compliance with cybersecurity requirements to the loss of customer trust. One prominent insider attack occurred when two employees of General Electric (GE) stole data on advanced computer models for calibrating turbines the company manufactured, as well as some marketing and pricing information. One of the employees even started a new company and competed with GE. In 2020, after several years of investigation, the insiders were convicted and sentenced to prison time and \$1.4 million in restitution to GE.

Originally, attackers were mostly individuals. However, more recent attacks have involved groups of people, particularly organized crime groups, launching attack campaigns. The motivation can range from fun or fame, to financial benefits, to political reasons. Some have very low technical skills, and others are highly skilled.

There are four main types of attackers with different motivations, as shown in the following table:

	Target	Goal	Expertise	Damage
Cyberterrorists	Critical infrastructures at the national or state level	Political advantage or social pressure	Mostly high, use sophisticated cyberattacks	Extreme or High
Cybercriminals	Sensitive information that can be sold or direct access to money	Financial gain	Variable	High
Hactivists	Companies and organizations that are an affront to their religion, politics, or cause	Political or social	High	Medium
Script Kiddies	Networks or websites with minimal security	Fun, fame	Low, mostly utilize scripts or programs developed by others	Small

Here are a few famous adversary groups (also called threat actors):

- Anonymous: decentralized hacking group primarily known for distributed denial-of-service (DDoS) cyberattacks. Their targets are political figures, foreign governments, and religious groups, or other entities that they view as corrupt, such as child pornography sites.
- Lizard Squad: a black hat hacking group that mainly targets gaming companies and websites.
- Wizard Spider: a Russian-based eCrime group that previously focused on profitable banking Trojans such as TrickBot, and now has shifted to ransomware.
- Lazarus Group: a group that originated in Pyongyang, North Korea and is known for the 2014 Sony Pictures hack.
- Fancy Bear: a Russian hacking group targeting foreign governments, embassies, media companies, and defense organizations. They were responsible for the hack of the Democratic National Convention that happened prior to the 2016 U.S. presidential election.

Attacks

Attackers employ a variety of tactics and techniques to achieve their goals. The [MITRE ATT&CKTM](#) framework provides a very useful taxonomy of attacks' tactics, techniques, and procedures (TTPs). It breaks intrusions into a series of 14 tactics that adversaries may employ, each with a number of different techniques that have been observed. Here are the 14 enterprise tactics:

- Reconnaissance: the adversary is trying to gather information they can use to plan future operations.
- Resource Development: the adversary is trying to establish resources they can use to support operations.
- Initial Access: the adversary is trying to get into your network.
- Execution: the adversary is trying to run malicious code.
- Persistence: the adversary is trying to maintain their foothold.
- Privilege Escalation: the adversary is trying to gain higher-level permissions.
- Defense Evasion: the adversary is trying to avoid being detected.
- Credential Access: the adversary is trying to steal account names and passwords.
- Discovery: the adversary is trying to figure out your environment.
- Lateral Movement: the adversary is trying to move through your environment.
- Collection: the adversary is trying to gather data of interest to their goal.
- Command and Control: the adversary is trying to communicate with compromised systems to control them.
- Exfiltration: the adversary is trying to steal data.
- Impact: the adversary is trying to manipulate, interrupt, or destroy your systems and data.

Under each tactic, a number of different techniques can be applied. Some techniques can span more than one tactic as they can be used for different purposes. For example, phishing may be applied in both reconnaissance to gain more information and to gain initial access. Rootkits can be applied as a technique to avoid detection. Data encrypted for impact is a technique used for impact. Ransomware uses this technique to encrypt common user files in order to extract monetary compensation from a victim in exchange for decryption.

Malware

Malware, also known as malicious code, refers to any firmware or software that is intentionally included or inserted into a system for a harmful purpose. It is often used by attackers to cause extensive damage to data and systems or to gain unauthorized access to a network. Some malware may damage programs, delete files, or even reformat the hard drive. Others may gain access to sensitive data or even simply flood a network with traffic to make normal services unusable.

Based on the 2020 global threat report by CrowdStrike (CrowdStrike, 2020) (attached), about 50% of attacks in 2018 and 40% in 2019 leveraged malware directly. (They define malware attacks as those cases where a malicious file is written to disk and CrowdStrike Falcon detects an attempt to run that file.)

There are several common types of malware, such as computer viruses, worms, Trojan horses, spyware, adware, ransomware, rootkits, etc.

Ransomware

Ransomware is a type of malware designed to deny access to a computer system or data until a ransom is paid. It is basically a DoS attack. Ransomware typically spreads through phishing emails, instant messages, or by a victim unknowingly visiting an infected website. It can lock a system, or encrypt valuable files on a system so that the user cannot access them. The attacker then demands a ransom from the victim to restore access to the data upon payment.

The following table shows the largest ransom demands reported in 2019 according to the threat report by CrowdStrike. Ransomware was widespread and affected all sectors, including local governments, academia, technology, healthcare, manufacturing, financial services, and media.

**Largest Ransom Demands Reported
in 2019**

USD	BTC	Malware
\$12.5M	-1,600	Ryuk
\$10.9M	565	DoppelPaymer
\$10.0M	1,326	REvil
\$9.9M	1,250	Ryuk
\$6.1M	850	Maze

\$6.0M	763	REvil
\$5.3M	680	Ryuk
\$2.9M	375	DoppelPaymer
\$2.5M	250	REvil
\$2.5M	250	DoppelPaymer
\$2.3M	300	Maze
\$1.9M	250	DoppelPaymer
\$1.6M	216	BitPaymer
\$1.0M	128	Maze

Ryuk was one of the first ransomware families to identify and encrypt network drives and resources and delete shadow copies on the endpoint. The following screenshot shows a typical message displayed by Ryuk.

```

Your network has been penetrated.

All files on each host in the network have been encrypted with a strong algorithm.
Backups were either encrypted or deleted or backup disks were formatted.
Shadow copies also removed, so F8 or any other methods may damage encrypted data but not recover.
We exclusively have decryption software for your situation
No decryption software is available in the public.

DO NOT RESET OR SHUTDOWN - files may be damaged.
DO NOT RENAME OR MOVE the encrypted and readme files.
DO NOT DELETE readme files.
This may lead to the impossibility of recovery of the certain files.

To get info (decrypt your files) contact us at
AndyMitton@protonmail.com
or
AndyMitton@tutanota.com

BTC wallet:
1LKULheYnNTjXgQNwMo24MeLrBBCouECH7

Ryuk
No system is safe

```

The windows system restore can be disabled making recovery impossible without external backups. Ryuk has targeted high-profile organizations such as EMCOR, UHS hospitals, and several newspapers.

Ransomware attacks are one of the most commonly observed attacks and have gained a lot of attention in the last couple of years. Targeted, criminally motivated, enterprise-wide ransomware attacks will continue or even escalate in the future with the development of RaaS (ransomware-as-a-service).

Trojan Horses

The Trojan Horse is a story from the Trojan War about the subterfuge that the Greeks used to enter the independent city of Troy and win the war. This terminology was first used to describe malware in a 1974 U.S. Air Force report on the analysis of vulnerability in computer systems. It refers to any malware that misleads users of its true intent. It was made popular by Ken Thompson in his 1983 Turing Award acceptance lecture, "Reflections on Trusting Trust: To what extent should one trust a statement that a program is free of Trojan horses? Perhaps it is more important to trust the people who wrote the software."

A Trojan horse appears to be benign (e.g., a game, an attachment, or a legitimate application), but contains hidden and malicious code. It relies on the user to download it through spam, phishing, advertising, drive-by-downloads, or social engineering, and to run it on the target.

Banking Trojans are a specific kind of Trojan malware. Once installed, a banking Trojan can allow outside attackers to gain access to computer files and systems, make unauthorized transactions, steal clients' identities, or transfer client funds to attackers' accounts.

Dridex, a well-known banking Trojan that first appeared in 2012, became one of the most prevalent malware programs in 2020. Dridex is a sophisticated strain of banking malware that targets the Windows platform, delivering spam campaigns to infect computers and steal banking credentials and other personal information to facilitate fraudulent money transfers. The malware has been systematically updated and developed over the past decade. Dridex is now being used in the early attack stages for downloading targeted ransomware, such as BitPaymer and DoppelPaymer.

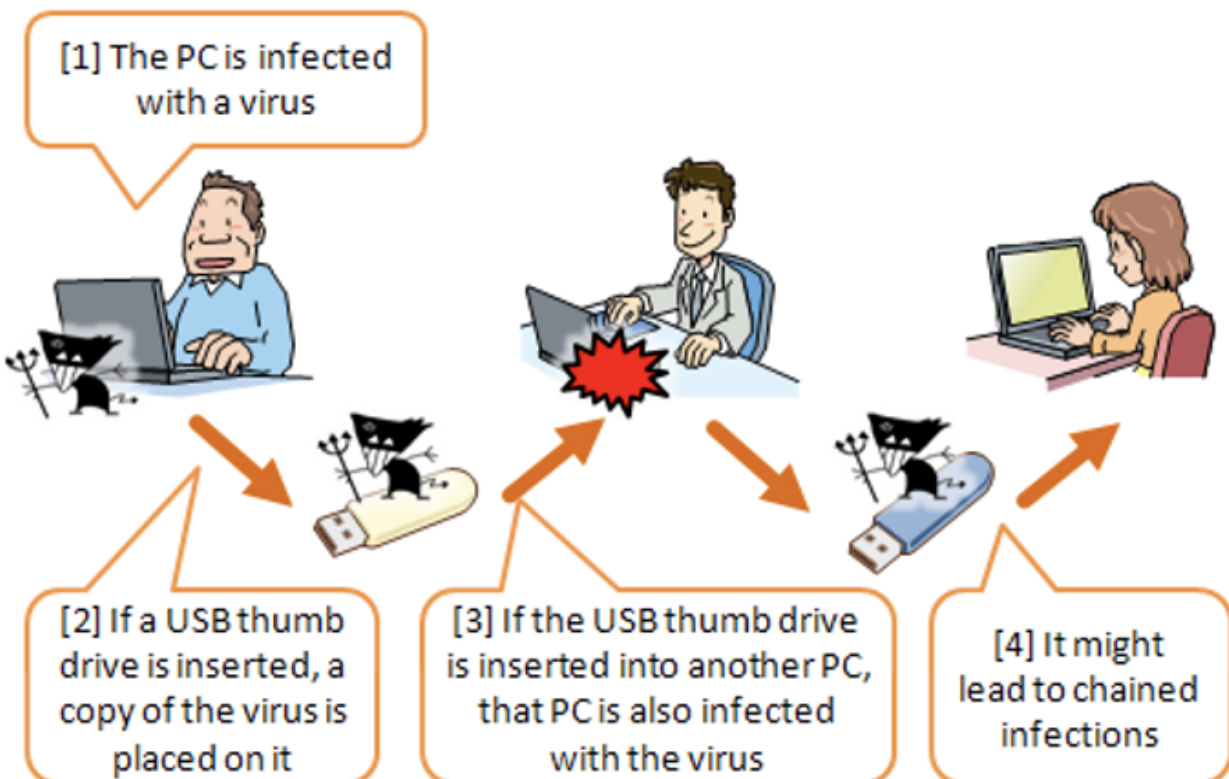
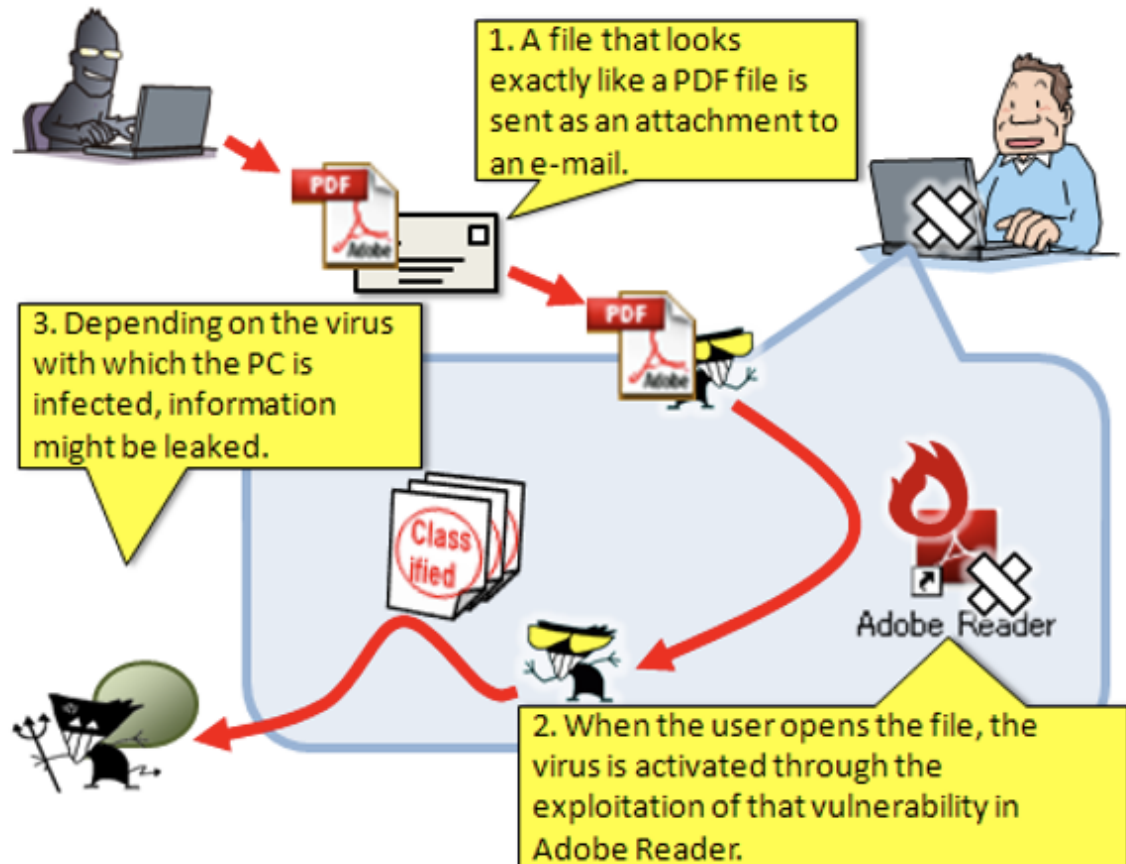
Computer Viruses

A computer virus is a piece of code that can replicate itself and pass on malicious code to other non-malicious programs by modifying them. Similar to biological viruses, computer viruses infect a healthy subject by attaching themselves to it. The infected program then acts as a virus to infect others.

A virus can be resident or non-resident. A non-resident virus runs when the infected program executes and terminates when the infected program ends. But a resident virus will remain in RAM even after its infected program ends. It usually installs itself as part of the operating system and runs from the time the computer is booted up to when it is shut down.

A virus can affect not only executable programs, but also documents, such as Word, Excel, or PDF files. These document types allow scripts to run in order to generate some dynamic content. For example, Microsoft Word documents can contain some macro code. So, attackers can create a macro virus written in a macro language and insert it into a Word document. When the user opens the document, the virus code is executed, and can then further infect the user's computer. The infected documents are often sent as email attachments. The virus may then harvest email addresses from various sources on the infected machine and send copies of itself to all addresses.

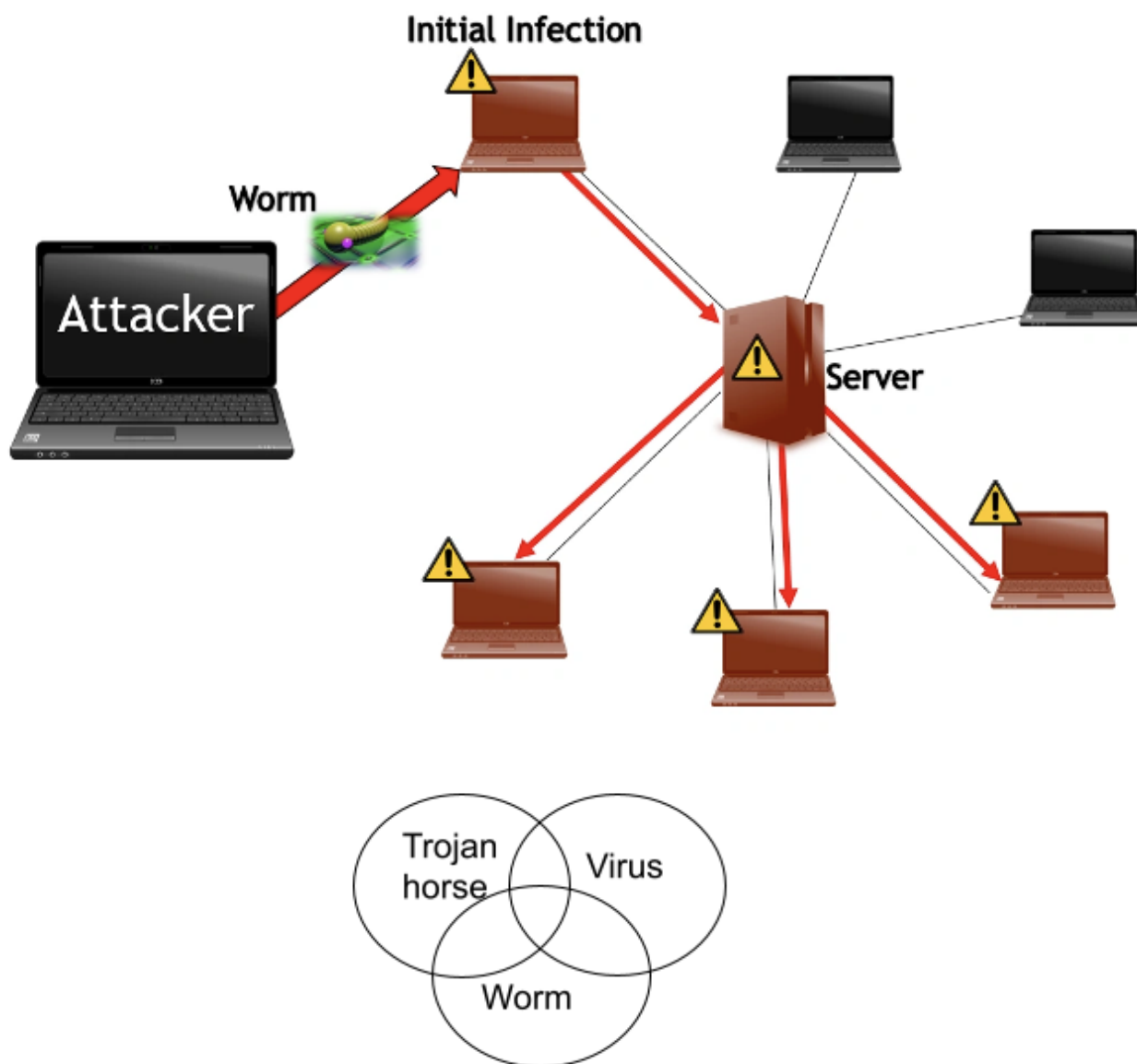
You may be familiar with antivirus (AV) software such as Norton and McAfee. They were initially developed to detect and remove computer viruses, particularly on Windows platforms. Now, most AV software target not only viruses, but also general malware.



Computer Worms

The terms worm and virus are often used interchangeably, but they are different. A worm is a piece of malicious code that usually searches the network for similarly vulnerable hosts to propagate itself. Similar to a virus, a worm can replicate itself quickly and infect others. However, unlike viruses, which can spread through any medium, worms spread through a network. Worms usually cause some harm to the network, even if only by consuming bandwidth, whereas viruses usually corrupt or modify files on a targeted computer. Another difference is that a worm is not embedded in other programs; rather it runs as an independent process. To avoid being detected, a computer worm often disguises its process name or info to hide its presence, so that it cannot be easily detected in the task manager.

The famous Morris worm was created by Robert Tappan Morris, a graduate student at Cornell University, and launched on November 2, 1988. It quickly spread across the internet, disrupting many computers then on the internet. Morris was the first convicted felon in the U.S. under the 1986 Computer Fraud and Abuse Act.

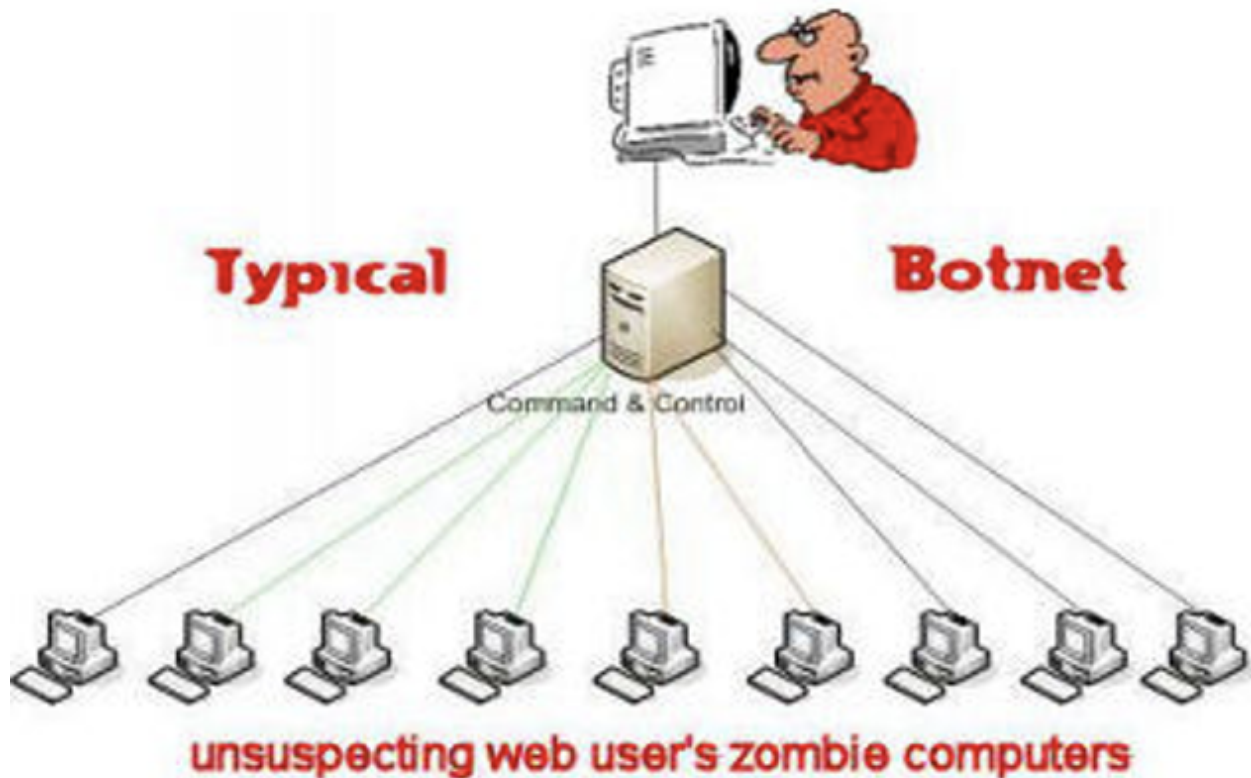


Zombies/Bots and Botnets

A zombie, also known as a “bot,” is a computer remotely controlled by attackers to perform attack activities, such as spreading viruses or worms and sending spams, on other computers on the internet. A bot needs to be internet-connected and computing capable. Most owners of zombie computers are not aware of it!

Attackers typically exploit multiple computers to create a botnet (basically a collection of bots), also known as a zombie army. They are usually used for DDoS (distributed denial-of-service) attacks, emailing spam (such as sending junk emails), and mining bitcoins.

The notable Mirai botnet attack in 2016 was a massive DDoS attack that disrupted the internet on the East Coast of the U.S.. Mirai is a malware that turns vulnerable networked devices running Linux into remotely controlled bots. It particularly targets insecure IoT devices such as IP cameras and home routers. The Mirai botnet was created initially to perform DDoS attacks on Minecraft servers, and later contributed to several large-scale DDoS attacks, including the September 20, 2016 attacks on the KrebsOnSecurity website.



Rootkits

A rootkit is a set of software tools that enable an unauthorized user to gain control of a computer system without being detected. It usually has two capabilities. One is to gain root access; the other is to hide itself. Therefore, it is very dangerous and difficult to detect. It usually masks itself from detection by anti-malware scanners, and can even subvert detection programs.

There are several types of rootkits—kernel rootkits, user-mode rootkits, bootloader rootkits, memory rootkits, and firmware rootkits—which differ based on where the rootkits are injected and what they target. Kernel rootkits work at the level of the operating system kernel, and can add, modify, or delete OS kernel code. Bootloader rootkits attach themselves to the boot records such as the MBR (master boot recorder) and are much harder to detect and remove. User-mode rootkits are simpler and easier to detect than kernel or bootloader rootkits. This is because they hide within an application itself, and not within critical system files. Memory rootkits hide in the memory (RAM) and will disappear once the system reboots. Firmware (or even hardware) rootkits intend to infect firmware (or hardware) such as hard drives, routers, network cards, or even a BIOS (Basic Input Output System). These types of rootkits are rare and harder to detect. Researchers at Kaspersky discovered one such rootkit, which compromised the UEFI (Unified Extensible Firmware Interface), and targeted African, Asian, and European diplomatic and nongovernmental organizations between 2017 and 2019. The rootkit deploys malware into a target directly and executes it when the system boots up.

Another example is the infamous Sony BMG scandal in 2005. Sony BMG inserted a rootkit on about 22 million CDs. When the CD was inserted into a computer, a music player program was installed on the computer together with a silent rootkit. While the software was intended to provide a form of digital rights management (DRM), it was added by modifying the operating system to interfere with CD copying. The program hid itself, which made it hard to detect and uninstall. It also created vulnerabilities that could be exploited by malware.

Spyware & Adware

Spyware is a type of malware that collects the information of users without their knowledge and transmits the data back to a remote entity, often for financial gain. Adware is a type of software that automatically delivers advertisements. Common examples of adware include pop-up ads on websites and advertisements that are displayed by software. Many “free” versions of applications are bundled with adware in order to generate some revenue. In order to deliver targeted advertisements, adware is often bundled with spyware that is capable of tracking user activity and stealing information. Due to the added capabilities of spyware, adware/spyware bundles are significantly more dangerous than adware on its own.

Hiddad is a family of hidden adware. It generates revenue through aggressive advertising, and conceals itself to prevent it from being uninstalled. Hiddad malware typically takes the form of a legitimate app, such as a QR code reader or an image editing app. Numerous apps riddled with Hiddad malware have been discovered over the past year. SophosLabs reports that it has identified at least 57 Hiddad apps with a total of 15 million unique installations from Google Play in September 2019 alone, and a new set of such apps appeared every few weeks.

Vulnerability Taxonomy

Attacks usually occur when threat sources such as malware, attempt to exploit vulnerabilities. Vulnerabilities are weaknesses in information systems, system security procedures, internal controls, or their implementation that

could be exploited or triggered by threat sources.

There are thousands of known vulnerabilities, with new vulnerabilities discovered and exposures published every week. [Common Weakness Enumeration \(CWE\)](#) is a community-developed list of software and hardware weaknesses that can occur in architecture, design, code, or implementation and can lead to exploitable security vulnerabilities. [Common Vulnerability Enumeration \(CVE\)](#) is a list of publicly known cybersecurity vulnerabilities and exposures. Both lists are maintained by MITRE Corp.

The CVE List was launched by MITRE as a community effort in 1999, and the U.S. National Vulnerability Database (NVD) was launched by the National Institute of Standards and Technology (NIST) in 2005 as a vulnerability database built upon and fully synchronized with the CVE List so that any updates to CVE appear immediately in NVD. NVD provides enhanced information for each record such as fix information, severity scores, and impact ratings, as well as advanced searching features such as by OS, vendor name, product name, version number, vulnerability type, severity, related exploit range, and impact.

Each CVE record has an identifier, such as "CVE-2020-8911." A vulnerability can be assessed using CVSS (The Common Vulnerability Scoring System) to calculate its severity or prioritize its remediation activities. The base metric is a score ranging from 0 to 10, which can then be modified by scoring the temporal and environmental metrics. [The National Vulnerability Database \(NVD\)](#) provides CVSS scores for almost all known vulnerabilities. Here is the rating scale defined in CVSS v3.0.

Severity	None	Low	Medium	High	Critical
Base Score Range	0.0	0.1-3.9	4.0-6.9	7.0-8.9	9.0-10.0

A sample CVE report is shown below.

CVE-2020-0354 Detail

Current Description

In Bluetooth, there is a possible out of bounds write due to a missing bounds check. This could lead to remote code execution with no additional execution privileges needed. User interaction is not needed for exploitation. Product: Android Versions: Android-11 Android ID: A-143604331

[+View Analysis Description](#)

Severity

CVSS Version 3.x

CVSS Version 2.0

CVSS 3.x Severity and Metrics:



NIST: NVD

Base Score: 9.8 CRITICAL

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

NVD Analysts use publicly available information to associate vector strings and CVSS scores. We also display any CVSS information provided within the CVE List from the CNA.

Note: NVD Analysts have published a CVSS score for this CVE based on publicly available information at the time of analysis. The CNA has not provided a score within the CVE List.

Vulnerabilities can lead to violations of different aspects of CIA and IAAA, including:

- Poor or missing authentication, such as poor password selection
- Missing authorization, such as improper file permission or improper access control
- Improper or insufficient auditing
- Confidentiality violations, for example sensitive data, such as credentials, stored or transmitted in clear text
- Integrity violation due to a lack of proper authentication and access control
- Availability violation, such as DOS (denial of service), due to lack of proper authentication

Vulnerabilities can be in different components of a system such as hardware, software (operating systems or user-level applications), network protocols, language design, configuration, and users.

Vulnerabilities in Hardware

The famous [Meltdown and Spectre](#) attacks exploit critical vulnerabilities in modern processors in order to steal sensitive data in the memory, such as passwords stored in a password manager or browser. All modern processors capable of keeping many instructions in flight are potentially vulnerable. **Meltdown** exploits the side effects of out-of-order execution on modern processors to read arbitrary kernel-memory locations. It breaks the most fundamental isolation between user applications and the operating system. **Spectre** exploits a race condition vulnerability in the design of the speculative execution implemented in modern processors to read the data in the restricted area that should be inaccessible. Spectre is harder to detect than Meltdown, and it is also harder to mitigate.

Detection can be as hard as exploitation, as these attacks do not leave any traces in traditional log files. It is unknown if they have been abused in the wild or not. Fortunately, there are software patches against these attacks.

Another example of a hardware vulnerability is [the “rowhammer” bug](#). In some recent dynamic random access memory (DRAM) devices, repeatedly accessing a row of memory can cause bit flips in adjacent rows. This can be exploited to gain kernel privileges. Due to the high cell density in modern DRAM, the isolation between DRAM cells can be violated by specially crafted memory access patterns that rapidly access the same memory rows numerous times. A rowhammer attack can cause a cell to leak charge to change its content within a refresh interval.

On March 9, 2015, Google's Project Zero revealed two working privilege escalation exploits based on the rowhammer effect, establishing its exploitable nature on the x86-64 architecture.

Vulnerabilities in the Software

Software is everywhere. Most attacks occur through exploitation of vulnerabilities in software. Gary McGraw, the author of “Software Security, Building Security In,” argued that half of software vulnerabilities are design flaws and half are implementation bugs.

The term **“bug”** was first used in an account by computer pioneer Grace Hopper, who publicized the cause of a malfunction in an early electromechanical computer. Operators traced an error in the Mark II to a moth trapped in a relay, thus inspiring the term bug. This bug was carefully removed and taped to the log book. Now the term bug is usually used to describe an error made in the code.

There are many types of vulnerabilities in software. The CWE Top 25 list identifies the 25 most widespread and critical weaknesses that can lead to serious vulnerabilities in software.

The CWE Top 25 2022

1. Out-of-bounds Write
2. Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
3. Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
4. Improper Input Validation
5. Out-of-bounds Read
6. Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')
7. Use After Free
8. Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')
9. Cross-Site Request Forgery (CSRF)
10. Unrestricted Upload of File with Dangerous Type
11. NULL Pointer Dereference
12. Deserialization of Untrusted Data
13. Integer Overflow or Wraparound
14. Improper Authentication
15. Use of Hard-coded Credentials

16. Missing Authorization
17. Improper Neutralization of Special Elements used in a Command ('Command Injection')
18. Missing Authentication for Critical Function
19. Improper Restriction of Operations within the Bounds of a Memory Buffer
20. Incorrect Default Permissions
21. Server-Side Request Forgery (SSRF)
22. Concurrent Execution using Shared Resource with Improper Synchronization ('Race Condition')
23. Uncontrolled Resource Consumption
24. Improper Restriction of XML External Entity Reference
25. Improper Control of Generation of Code ('Code Injection')

The OWASP (The **O**pen **W**eb **A**pplication **S**ecurity **P**roject) Top 10 lists the 10 most common software vulnerabilities in web applications.

OWASP Top 10

- A1: Injection
- A2: Broken Authentication and Session Management
- A3: Sensitive Data Exposure
- A4: XML External Entities (XXE)
- A5: Broken Access Control
- A6: Security Misconfiguration
- A7: Cross-Site Scripting (XSS)
- A8: Insecure Deserialization
- A9: Using Components with Known Vulnerabilities
- A10: Insufficient Logging and Monitoring

The OWASP Mobile 10 lists the 10 most common software vulnerabilities in mobile applications.

OWASP Mobile Top 10

- M1: Improper Platform Usage
- M2: Insecure Data Storage
- M3: Insecure Communication
- M4: Insecure Authentication
- M5: Insufficient Cryptography
- M6: Insecure Authorization
- M7: Client Code Quality

M8: Code Tampering

M9: Reverse Engineering

M10: Extraneous Functionality

Operating systems are the lowest-level software that is critical to a system. Vulnerabilities in the operating system kernel usually result in more severe attacks that can take over the whole system. Kernel mode rootkits often exploit vulnerabilities in the OS kernel. Vulnerabilities in software such as compilers, web browsers, and virtual machine monitors also have more severe impacts than normal user-level applications.

Vulnerabilities in Network Protocols

Most network protocols were designed initially without security in mind. Many protocols lack authentication, such as IPv4 and ICMP, ARP, tFTP, and DNS. Others only have very weak authentications, such as BGP, PPP, HTTP, TCP, LDP, SNMP, and NTP.

For example, ARP (Address Resolution Protocol) is a simple protocol used to discover the media access control (MAC) address of a host given the corresponding IP address on Ethernets. A request with an IP address is broadcast on the network segment, and the addressed entity replies with a response containing its MAC address, which the querying entity then caches for several minutes. No authentication is used whatsoever. This makes ARP spoofing very easy. The attacker can associate their machine's MAC address with the IP address of the target so that any traffic meant for the target is received by the attacker.

Many network protocols don't consider confidentiality and integrity either. Data is transmitted in "clear" text without any encryption, and can be easily intercepted. For example, Telnet is an application protocol above TCP/IP for text-oriented communication. All messages are sent in clear text. HTTP (Hypertext Transfer Protocol) is an application-layer protocol above TCP/IP for transmitting hypermedia documents, such as HTML. It is also not secure because all HTTP connections are not encrypted. Additional secure protocols such as IPsec, DNSsec, SSL, SSH, and HTTPS need to be adopted to mitigate these security issues.

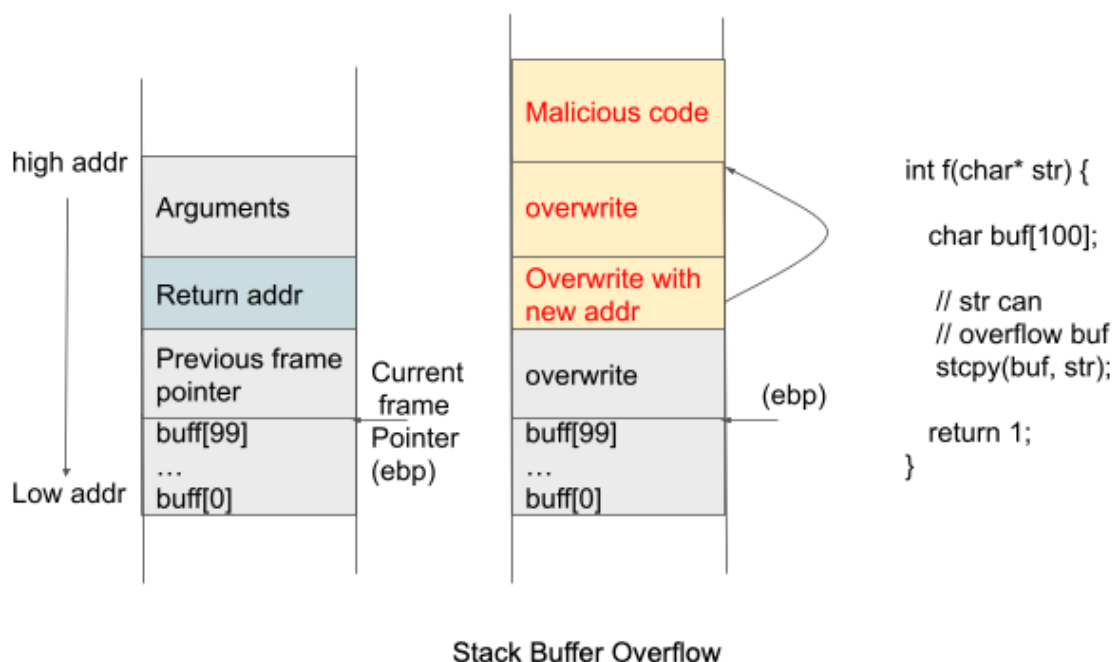
Vulnerabilities in Programming Languages

Software is written in certain programming languages. Vulnerabilities related to a programming language can affect all software written in that language. C is a very powerful and flexible programming language developed in the 70s that is designed to provide low-level access to memory and language constructs that map efficiently to machine instructions. It is one of the most popular programming languages, and is particularly used in the development of lower-level software such as operating systems kernels (Linux), compilers, etc., due to its high performance. Many later programming languages borrow directly or indirectly from C.

However, a number of security vulnerabilities are associated with C. Buffer overflow is one of the most famous and severe ones. A buffer overflow occurs when a program attempts to put more data in a buffer than it can hold.

The stack buffer overflow exploit, in which the attacker overflows the stack buffer in order to overwrite the return pointer, is the most common type of buffer overflow exploit. The result is that when the function returns, it transfers control to malicious code contained in the attacker's input data.

The following diagram shows how a stack buffer overflow exploit can occur. The left figure shows the function stack, which contains the local variable `buf`, previous frame pointer, return address, arguments, etc. The right side shows a sample of vulnerable C code. The function call `strcpy()` copies the input parameter `str` into the `char` array `buf`. Since `buf` is only 100 bytes without any boundary checking, when the `str` is longer than 100 bytes it can overflow `buf`. The attacker can craft a special input argument `str`, so that it overwrites the original return address to the address of the malicious code that is injected into the stack. When the function `f()` returns, the malicious code will be executed.



Even though most software developers know what a buffer overflow vulnerability is, buffer overflow attacks against both legacy and newly-developed applications are still quite common.

Java is a type-safe language, and is more secure than C in general. However, there are also vulnerabilities in Java. For example, reflection allows a running Java program to examine or "introspect" upon itself, and manipulate internal properties of the program. An attacker can exploit unsafe use of reflection to create unexpected control flow paths through the application, potentially bypassing security checks.

Configuration Vulnerabilities

Security misconfigurations are one of the most common vulnerabilities. There are many examples, including:

- Using default or simple passwords.
- Leaving insecure service ports open (e.g. leave `tfnp` enabled).

- Writing the hostname, login, and password in a .netrc file for easy access.
- Using wide read-write export of a file system, or world-writable system files or directories, or incorrect file/directory permissions.
- Allowing hidden SUID shell files or unauthorized SUID files.
- Using .rhosts files to allow any users with a privileged user's loginID to open rlogin, rcp or rsh to access the host with that user's privilege.
- Only using HTTP without HTTPS.

User/Social Engineering

The greatest security weakness may be users themselves. Without user awareness, even the most secure technology may not be effective. Social engineering is the psychological manipulation of people into performing actions or divulging confidential information. It is a kind of “human hacking” strategy that exploits the weaknesses of human beings. Phishing attacks are one of the most common social engineering attacks. Attackers pretend to be a trusted institution or individual in an attempt to persuade users to expose their personal data and other valuables. Spam phishing is a widespread attack aimed at many users, while spear phishing uses personalized info to target particular users, usually high-value targets like celebrities, upper managers, and prominent government officials.

Test Yourself

Test Yourself

We only need to worry about attacks from outsiders.

True

False

Test Yourself

Which of the following groups usually has low expertise and attack for fun? (select one)

Cyberterrorists

Cybercriminals

Hactivists

Script Kiddies

Test Yourself

Briefly explain why a ransomware attack is a type of DoS attack.

Ransomware denies legitimate users from accessing their own systems or data until the ransom is paid, by locking a system, or mostly by encrypting valuable files on a system.

Test Yourself

_____ needs to attach to other programs. _____ can duplicate itself quickly. _____ mainly spreads through networks. _____ appears to be a legitimate software, but hides its original malicious intention. _____ can gain root access and hide itself.

- a. A trojan horse
- b. A virus
- c. A worm
- d. A rootkit

B, B & C, C, A, D

Test Yourself

Check your home router model and see if you can find a vulnerability related to that model from the CVE list.

Test Yourself

Examine if there is any configuration vulnerability in your home network and your computers.

 Topic 2 Defense

Defense

In the first section, we discussed various vulnerabilities and attacks. Now, let us discuss security principles and controls that help defend against those attacks.

Cryptography is the theoretical foundation of cybersecurity. It is the practice and study of techniques for secure communication in the presence of third parties called adversaries. It particularly addresses the confidentiality and integrity issues in data communication through various methods such as encryption, cryptographic hash, and digital signatures.

A firewall is another commonly used security control for information security. Through packet filtering, firewalls can prevent unauthorized access to or from a private network. They also address the availability issue in the CIA triad. Firewalls are the primary perimeter defense in most organizations.

Penetration testing has become a buzzword in recent years. It is a method for evaluating the security of a system and assessing its vulnerabilities through simulated attacks. The idea is to assess the system security from the attacker's point of view. Performing thorough penetration testing can help the organization identify a range of vulnerabilities, particularly known, high-risk ones. Kali Linux is a Debian-based Linux distribution aimed at advanced penetration testing and security auditing. It contains a large amount of penetration testing tools from various different niches of the security and forensics fields. In particular, Metasploit is included in Kali as a penetration testing platform to find, exploit, and validate vulnerabilities.

While cryptography, firewalls, and penetration testing do help mitigate some issues in cybersecurity, they are not magic bullets for addressing all problems and guaranteeing security. In fact, security cannot be simply added into the system, but needs to be built into it. The advancement of cloud computing, mobile computing, and IoT changed the traditional cybersystems. Cybersystems are now too complex, exposing too many attack surfaces. There is no silver bullet that can prevent all attacks and solve all security issues. We cannot just rely on the traditional network-based perimeter defenses such as firewalls, crypto, or secure protocols. The growing connectivity and extensibility make security problems even worse and harder to solve.

Security controls include both technical controls, such as cryptography, firewalls, logs, penetration testing, and secure coding, as well as non-technical controls, such as policies, procedures, laws, etc. **Defense in Depth** is a very important concept in which multiple layers of security controls (defenses) need to be applied throughout the system. The Defense in Depth principle is applied in various ways. For example, we need to defend against both outsider and insider attacks. We need to protect the system with multiple tools/mechanisms/techniques. We need to have security controls in different system components, including physical security, personnel security, network security, device security, application security, data security, infrastructure security, etc. We need to consider the defense from the perspective of different stakeholders, including:

- From the user's point of view
- From developer's point of view
- From designer's point of view

From the User's Point of View

Users are the end point of the system, and can be the greatest weakness as well. At the user level, it is very important to avoid installing and using/accessing malicious or vulnerable software/websites. Raising the security awareness of users is critical to a security program.

Here are a few defensive actions that should be used by users.

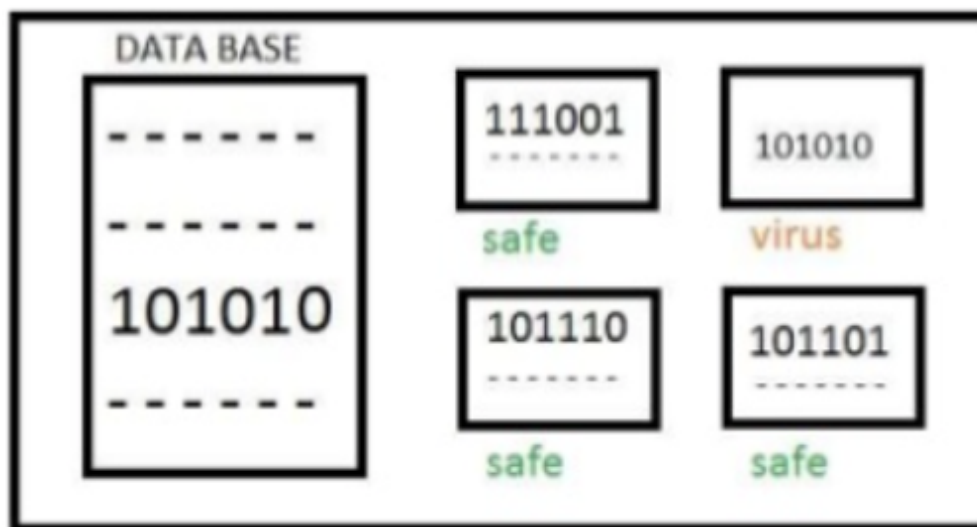
- Only install and use software from known, reliable vendors. This can help prevent malicious software from running on a user's computers. For example, only install a mobile app from the official app store, such as the Google Play or Apple stores. Install software on your computer from their official websites, instead of third-party websites. Sometimes, we may not be able to discern which vendors are reliable. Be diligent and do some research before installing a software. Web browser plugins and extensions are sometimes neglected by users and antivirus software, as the web browser is often considered trustable. However, those plugins and extensions are developed by third-party vendors. They run inside the web browser and thus can have the same privilege as the web browser to access information such as browsing history, website content, and even login credentials. Malicious or vulnerable plugins can cause severe security issues.
- Test software from unknown vendors in an isolated environment if you must use such software. An isolated environment can be a virtual machine or a machine that is separated from other systems or data, so that the possible damage, if any, can be contained.
- Make sure all software is up-to-date, particularly in terms of security updates. Be aware that benign software from reliable vendors can be vulnerable and exploited by attackers to launch attacks.
- Be very cautious when opening links and attachments in emails and chats. As mentioned previously, phishing attacks often occur when users are duped into opening an email, instant message, or text message.
- Recognize and avoid any websites that could be harmful. In particular, some malicious websites use very similar names to reputable websites such as "google.com" instead of "google.com." Users who incorrectly type a website address may be targeted by attackers when accessing those malicious websites. This is usually known as URL hijacking or typosquatting.
- Backup your system and data securely and periodically, particularly critical data. You can use either cloud storage or external drives to store backups. This is also an effective defense against most ransomware attacks.
- Install reputable and effective antivirus (AV) software if needed. Antivirus (or mostly anti-malware) software requires high privileges to access the system in order to detect and remove malware. Therefore, if such software itself is malicious or vulnerable, it can cause higher damage. It is important to install only reputable anti-malware software, not just any software that is declared to be so. Get familiar with the functionality of the anti-malware software and know what their expected behaviors should be. The following screenshot shows a common scam used to trick you into installing malware onto your computer, a sudden message that may pop up in your system or your browser.



Antivirus Software

There are usually two different methods for detecting viruses and malware. Signature-based detection is used to find specific patterns (signatures) inside code. The signatures are known in advance; therefore, this type of detection cannot detect zero-day malware which exploits unknown and unprotected vulnerabilities.

Signature-based detection is usually static before the malware is run.



Behavior-based detection examines malicious behaviors during runtime. It is usually dynamic during the running time. It can also predict generic malicious behaviors, and can be more effective for detecting zero-day malware. But there is latency, especially for a backdoor attack.

From the Developer's Point of View

As Gary McGraw argues in [Avoiding the Top 10 Software Security Design Flaws](#), half of security vulnerabilities are due to implementation bugs. Software developers should not only write workable and efficient code, but also secure code. Programmers should be aware of common software security bugs and try to avoid them. Various secure coding guidelines have been developed to address this issue. For example, [Oracle published secure coding guidelines for Java SE](#). OWASP published a [secure coding practices quick reference guide](#).

In addition, developers should use both SAST (Static Application Security Testing) and DAST (Dynamic Application Security Testing) tools to find security vulnerabilities. SAST is a set of technologies designed to analyze application source code (or byte code or binaries) statically to identify possible security vulnerabilities. DAST technologies are designed to assess security vulnerabilities in an application dynamically in its running state. DAST is mostly done by purposely injecting malicious data or faulty actions into an application to identify common security vulnerabilities, as in SQL injection and cross-site scripting. [SonarQube](#) is a SAST tool used by many organizations. It is a cross-language source code analysis tool for bugs, vulnerabilities, and code smells. It supports more than 20 languages and has IDE plugins for Eclipse, Visual Studio, and IntelliJ. Appspider by Rapid 7 and Burp Suite by Portswigger are DAST tools that support auto scanning for vulnerabilities. Arachni is an open-source security scanner.

From the Designer's Point of View

Besides implementation bugs, design flaws are another main source of security vulnerability, and are usually harder to identify than implementation bugs. The IEEE Center for Secure Design was created by the IEEE Computer Society in 2014. The foundational workshop gathered software security expertise from industry, academia, and government to discuss common design flaws and came up with [how to avoid the top 10 design flaws](#):

1. Earn or Give, but Never Assume, Trust
2. Use an Authentication Mechanism that Cannot be Bypassed or Tampered With
3. Authorize after You Authenticate
4. Strictly Separate Data and Control Instructions, and Never Process Control Instructions Received from Untrusted Sources
5. Define an Approach that Ensures all Data are Explicitly Validated
6. Use Cryptography Correctly
7. Identify Sensitive Data and How They Should Be Handled
8. Always Consider the Users
9. Understand How Integrating External Components Changes Your Attack Surface
10. Be Flexible When Considering Future Changes to Objects and Actors

Similarly, the [CSEC 2017 \(Cybersecurity Curricular Guideline\)](#) proposed 14 secure design principles in three categories.

1. Restrictiveness

- a. Least privilege: give just enough privileges to complete the task.
- b. Fail-safe defaults/fail securely: deny access by default unless explicitly granted.
- c. Complete mediation: validate every access.
- d. Separation (of domains/duties): should not grant access to a resource or take a security relevant action, based on a single condition.
- e. Minimize trust (Reluctance to trust): check all inputs and all security relevant actions.

2. Simplicity

- a. Economy of mechanism: security features should be as simple as possible.
- b. Least common mechanism: the sharing of resources should be reduced as much as possible.
- c. Least astonishment: security features of software, and security mechanisms they implement, should be designed so that their operation is as logical and simple as possible.

3. Methodology

- a. Open design: security of software, and of what that software provides, should not depend on the secrecy of its design or implementation.
- b. Layering: modules at a given layer interact only with modules in the layers immediately above and below it; reduce access points, enforcing the principle of separation.
- c. Abstraction: hide the internals of each layer, making only the interfaces available; this enables you to change how a layer carries out its tasks without affecting components at other layers.
- d. Modularity: software as a collection of cooperating components (modules); each module interface is an abstraction.
- e. Complete linkage: security design and implementation link to the security specification.
- f. Design for iteration: plan the design in such a way that it can be changed, if needed.

Security Modeling

To decide what security controls should be used and how to apply them, we need to first understand what we can trust and what are threats. Trust modeling and threat modeling are very important to security modeling.

Threat Modeling

As discussed in the previous module, a threat is any circumstance that has the potential to cause loss or harm. A threat model is a collection of threats that are deemed important for a particular environment. Threat modeling is the practice of identifying, enumerating, and mitigating and prioritizing potential threats. It answers questions such as “Where am I most vulnerable to attack?” “What are the most relevant threats?” and “What do I need to

do to safeguard against these threats?”. For example, if we want to protect data on servers, and identify that the biggest threat is remote attacks through networks, then network security should be the focus of the security control. If we assume that the biggest threat is physical attackers via break-in, then physical security is the key. Different threat models will result in different security controls.

The STRIDE model is a well-known approach to threat modeling introduced by Microsoft in 1999 and still widely used today. STRIDE stands for:

- Spoofing: a person, program, or system successfully identifies as another by falsifying data, to gain an illegitimate access.
- Tampering: illegitimate modification of data or a system to cause harm.
- Repudiation: Denial of the truth or validity of something.
- Information disclosure: illegitimate disclosure of information.
- Denial of service: denial of service for legitimate usage.
- Elevation of privilege: illegitimate elevated access to restricted resources by exploiting some vulnerabilities in the system.

The following table shows the mapping between the STRIDE model and the CIA/IAAA. The table also includes example threats in the Blackboard Learning System and possible mitigations.

STRIDE	CIA/AAA	Threat Examples in the Blackboard System	Mitigation
Spoofing	Authentication	Someone pretends to be another user	Passwords
Tampering	Integrity	Grades are modified by an attacker	Digital signature, Access control
Repudiation	Non-repudiation (Audit)	Student claims to have not posted a discussion thread	Logging, Digital signature
Information disclosure	Confidentiality (Data privacy)	Students' grades are viewed by another student	Encryption, Access control
Denial of service	Availability	Blackboard is not accessible	Load balancer, Elastic designer, Firewall
Elevation of privilege	Authorization	A student gets the privilege of an instructor	Sandbox roles, Access control, Firewall

Trust Modeling

Trust has been broadly studied in many different disciplines and used as the basis for decision making in diverse contexts. Trust refers to the degree to which an entity behaves as expected.

For example, if we choose to use third-party library code, we expect it can run correctly and reliably. Presumably we assume that it can provide the required functionality and will not crash our applications. But can we fully trust the third-party library code? The answer may be no. We know that there can also be vulnerabilities in these library codes that can be exploited by attackers. If an anti-malware application uses vulnerable library code, not only can it not be a reliable security solution, but it also exposes additional attack surfaces.

A trust model describes, for a particular environment, who is trusted to do what. Basically, we need to identify the set of software and data upon which the system depends for correct enforcement of security goals through trust modeling.

Recently, the Zero Trust model has been embraced as the cyber system becomes more complicated and attacks are more sophisticated. The original Zero Trust model of cybersecurity was developed by Forrester in 2010. The basic idea is that we should not automatically trust anything inside or outside of its perimeters and must verify anything and everything trying to connect to our systems before granting access.

Trusted Computing Base (TCB)

TCB is a combination of hardware, software, and controls that work together to form a trusted base to enforce your security policy.

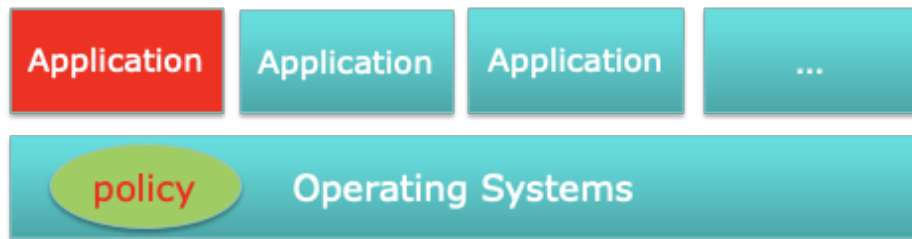
Think about the case of developing a malware detection solution. We need to decide where to deploy security policies and how to enforce them. A fundamental problem is, what do you trust?

The first solution is to deploy the policy in another application such as an anti-malware application, as shown in the following figure (a). This special application, acting as the police, attempts to monitor other applications and enforces the security rules. Here the assumption is that we can trust this special application.



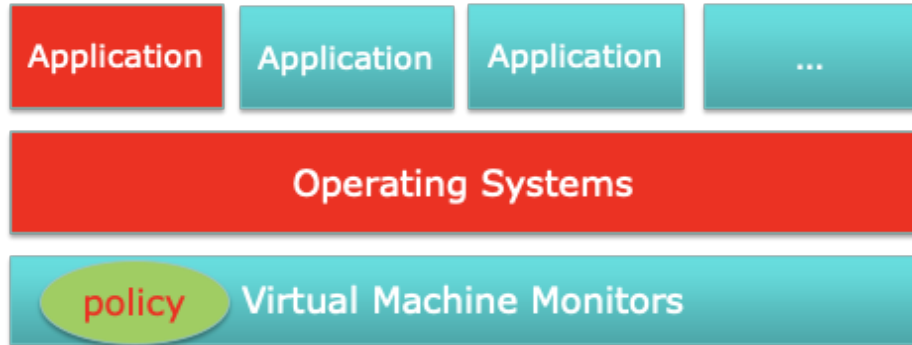
But if this application is compromisable, the above security control is in vain. Then we need to move our trust a layer down to the operating system as shown in figure (b). The operating system is, in general, more trustable than user-level applications. This solution can be more reliable than the previous one.

Security Policy Deployment Figure (b)

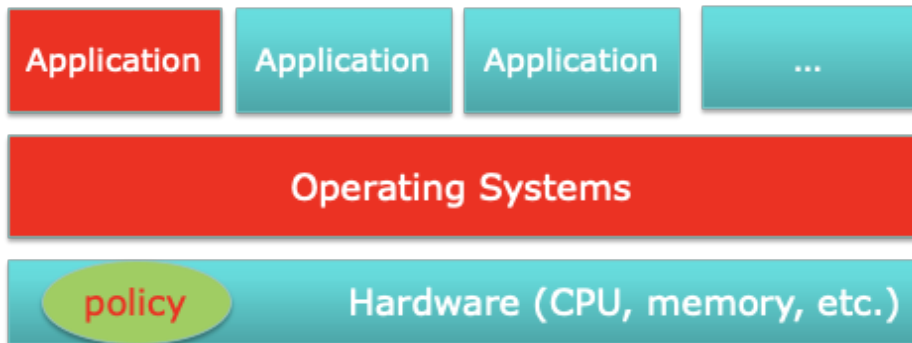


But it is also possible to compromise the operating system. Then we need to move the security enforcement even further down, i.e., to the virtual machine monitors, as shown in figure (c), or even to the lowest layer, hardware, as shown in figure (d).

Security Policy Deployment Figure (c)



Security Policy Deployment Figure (d)



Test Yourself

Test Yourself

Briefly explain Defense in Depth.

Multiple layers of security controls (defenses) need to be applied throughout the system.

Test Yourself

Many people say “the perimeter defense is dead.” Do you agree? Why?

Test Yourself

What are the differences between signature-based and behavior-based anti-malware?

Signature-based: mostly static; based on the specific characteristics of known malware. Behavior-based: mostly dynamic; looks for the possible signs of malicious behavior; can possibly detect zero-day attack.

Test Yourself

What are the pros and cons of SAST and DAST?

SAST pros: can check the source code; makes it easier to fix; detects in the early stage of SDLC.

Cons: needs to access the source code; can yield both false positives and false negatives.

DAST pros: can identify dynamic run-time vulnerabilities; checks the final product; doesn't yield false positives. Cons: occurs late in the SDLC; may not be easy to fix.

Test Yourself

List at least five important secure design principles.

Test Yourself

STRIDE stands for ____, ____, ____, ____, ____, and ____.

Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege